Interactive Visual Exploration of ODE Parameter Spaces through VAE Embeddings

Emiliano Islas and Noel Martin Poothokaran

May 9, 2026

1 Problem Description & Motivation

Dynamical systems are often modeled by ordinary differential equations whose form is abstractly represented as

$$\dot{x} = f(x; \theta), \tag{1}$$

where θ denotes a set of model parameters. In practice, the influence of individual parameters on the qualitative behavior of the solution is difficult to interpret, particularly when parameters interact nonlinearly. Variational Autoencoders (VAEs) can encode high-dimensional data into a low-dimensional continuous latent space, and have been shown to discover interpretable structure in an unsupervised manner. The primary contribution of this work is a visual analysis framework for parameterized dynamical systems. Rather than examining individual time-series plots, creating hard to read 3-D visualizations, or conducting parameter sweeps by repeatedly solving the ODE, the trained VAE provides an interactive low-dimensional map in which the user can visually identify how trajectories cluster, separate, and transition as parameters change. This approach transforms parameter exploration from a computational task—solve, plot, compare—into a navigable visual space where the relationship between parameters and dynamics can be perceived at a glance. Our tool also generalizes to high-dimensional situations such as a large number of parameters θ while retaining visual interpretability.

2 Related Work

Our work draws on three lines of research: latent variable models for dynamical data, visual parameter space analysis, and interactive exploration of learned embeddings.

Variational Autoencoders. Kingma and Welling [1] introduced the VAE as a generative model that couples an encoder–decoder architecture with a principled variational inference objective. The continuous latent space and the existence of a decoder distinguish the VAE from non-parametric projections such as t-SNE [2], making it possible to both visualize and generate data. We adopt the VAE as the core representation learning component of our framework to enable data point interpolation.

Neural ODEs and Latent Dynamical Models. Chen et al. [3] showed that continuous-depth residual networks can be interpreted as ODEs, establishing a formal bridge between deep learning and dynamical systems. Rubanova et al. [4] extended this idea by combining a VAE with an ODE-based latent dynamics model for irregularly sampled time series. While their goal is prediction and interpolation, our objective is interactive visual exploration of the mapping from parameters to qualitative dynamical behavior.

Visual Parameter Space Analysis. Sedlmair et al. [5] proposed a conceptual framework for visual parameter space analysis, systematizing how analysts explore the relationship between input parameters and model outputs through visualization. Our work instantiates this framework using learned latent representations rather than direct sampling grids or simple projection methods, enabling scalable exploration of high-dimensional parameter spaces.

Interactive Exploration of Learned Embeddings. Liu et al. [6] demonstrated interactive visual exploration of neural word embeddings, linking latent-space structure to semantic relationships. Their coordinated-view design informs our interface, though we operate in a fundamentally different domain—ODE trajectories rather than word vectors.

Dimensionality Reduction for Visualization. t-SNE [2] and UMAP [7] are widely used to project high-dimensional data to two dimensions for visual inspection. We included these methods as baseline to reference during development; however, because they lack a decoder, they cannot support the generative trajectory exploration that is a key feature of our system.

Interactive Latent Space Visualization Tools. Taylor Denouden's VAE Latent Space Explorer [9] provides a compelling demonstration of interactive latent space navigation for image data: users click within a two-dimensional latent grid and observe decoded outputs in real time, making the structure of the learned manifold directly perceptible. Our generative interpolation panel draws inspiration from this interface, adapting the click-to-decode paradigm from image generation to the visualization of ODE trajectories. Whereas the VAE Latent Space Explorer displays reconstructed images on a static grid, our system embeds the interaction within a coordinated multiple-view environment and

links decoded trajectories to parameter-space and time-series views, enabling analytical reasoning about dynamical behavior rather than passive inspection.

3 Technical Approach

Our framework consists of four stages: data generation, representation learning, interactive visualization, and scalability support.

3.1 Data Generation

We select canonical parameterized ODE systems that exhibit rich qualitative transitions as parameters vary. Benchmark systems include the FitzHugh–Nagumo model (spanning excitable and oscillatory regimes) and the Lotka–Volterra predator–prey model (spanning variation in oscillation frequency and equilibrium structure). For each system, we define a one-dimensional parameter sweep over a physically motivated range, holding all other parameters fixed so that inter-trajectory variation is attributable solely to the varied parameter. Each parameter value is numerically integrated using an adaptive Runge–Kutta 4/5 solver with tight tolerances to produce a long reference trajectory.

To construct the training dataset, each trajectory is segmented into fixed-length windows of size m via a sequential sliding procedure: windows are extracted at regular intervals with a small shift between consecutive windows, yielding multiple overlapping samples per trajectory. Each window of m time steps and k state variables is flattened into a feature vector $\mathbf{x} \in \mathbb{R}^{mk}$ and standardised per feature (zero mean, unit variance) across the full dataset. This windowing strategy ensures dense, uniform coverage of the attractor without requiring random sampling, and the shared initial conditions across parameter values ensure that observed variation reflects parameter effects rather than transient dynamics. Crucially, the window size m can be chosen to keep the input dimension mk tractable independently of the total trajectory length.

3.2 VAE Architecture and Training

A Variational Autoencoder is trained to map trajectory feature vectors to a low-dimensional latent space $\mathbf{z} \in \mathbb{R}^d$, with $d = 2$ or $d = 3$ for direct visual mapping. The encoder $q_\phi(\mathbf{z} \mid \mathbf{x})$ and decoder $p_\psi(\mathbf{x} \mid \mathbf{z})$ are parameterized as fully connected networks. Training minimizes the standard evidence lower bound (ELBO) loss:

$$\mathcal{L}(\phi, \psi) \;=\; -\,\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\big[\log p_\psi(\mathbf{x} \mid \mathbf{z})\big] \;+\; \mathrm{KL}\big(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, p(\mathbf{z})\big). \tag{2}$$

To encourage disentangled latent dimensions that align with interpretable dynamical properties, we experiment with the β-VAE variant [8], which up-weights the KL term. We also explore conditioning the decoder on θ so that the latent space captures variation beyond what the parameters already explain.

3.3 Interactive Visual Analytics Interface

We implement a web-based interface using D3.js and `onnxruntime-web`, served as a static single-page application. The interface is organised around a per-model panel comprising two coordinated views:

- **Latent Space View.** A two-dimensional scatterplot of the encoded posterior means $\boldsymbol{\mu} \in \mathbb{R}^2$ for a representative subset of training trajectories. Points are coloured by the varied parameter value using a per-model colour ramp, making parameter-to-geometry correspondence immediately legible. Clicking any point in the empty latent space triggers a live decoder inference at the selected (z_1, z_2) coordinate, with the reconstructed trajectory displayed in the detail view without a server round-trip.
- **Trajectory Detail View.** Displays the time-series of both state variables for the selected sample or decoder query, after denormalising with the per-feature statistics $\bar{\mathbf{x}}$ and $\hat{\sigma}$ exported alongside the model weights.
- **Parameter Effects View**. Two plots that focus on highlighting the effects of the selected parameter on the amplitude and phase of a signal over a range of its swept values.

All views are linked: selecting a training sample in any panel cross-highlights the corresponding point in all others and updates the trajectory detail views. The governing equations for the active system are rendered inline via MathJax, with the varied parameter typeset distinctively to reinforce the mapping between the physical parameter and the latent axis. Each model's JSON data and ONNX decoder session are lazy-loaded on first selection and cached for instant re-switching.

Unique Selling Point. While most visualizations online focus on a static set of input values and parameters for which they show a single set of trajectories, our visualization can display an essentially infinite (thanks to in-browser VAE interpolation) amount of trajectories for varied parameter values, allowing for exploration of un-tested parameters. We also enable the display of the trajectories (along with phase and amplitude) for the given-data and inferred values, a 2-D latent space view which visualizes hard-to-digest system dynamics. We also demonstrate that this pipeline can work for multiple different ODEs even as the number of parameters and equations scale up.

3.4 Scalability to Multiple ODE Systems

Because the VAE encoder maps trajectory windows $\mathbf{x} \in \mathbb{R}^{mk}$ to a fixed two-dimensional latent space regardless of the underlying ODE, the same frontend visualiser and ONNX inference pipeline apply uniformly across all registered models. Adding a new ODE or parameter configuration requires only a single registry entry in each of the three pipeline scripts; the architecture, training loop, export routine, and visualizer are unchanged.

3.5 Visualization

Figure 1: Main visualization displaying the 3 sets of views: Latent space, Parameter Effects, and Trajectories in left to right order.

Under Figure (1) we observe our main view for one of the equations and a parameter (in yellow) sweep. We also observe the latent space view, which allows us to identify a common direction for our change in paramater, e.g. we can observe that as the circles get smaller the value of alpha decreases somewhat linearly, which tells us that there is a direction of the parameter that is being encoded. In the parameter effects view in the model, we can see a similar linear trend where the change in alpha seems to be positively correlated to amplitude, so it is likely that our latent space encoded that as the direction towards the middle. The rightmost plot allows us to see those effects first-hand through the trajectories selected.

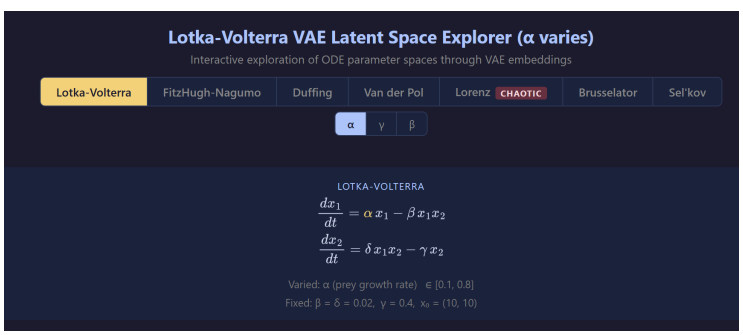

Figure 2: A closer look at the equation and parameter selection menu

Interactions allow for the clicking of the top buttons which changes to different equations and parameter combinations, we can also select points in the

latent space as to observe how its trajectory changes in the trajectory view, and also we can select spaces in the latent space without points as to infer a trajectory based on the encoding learned by the VAE 3.

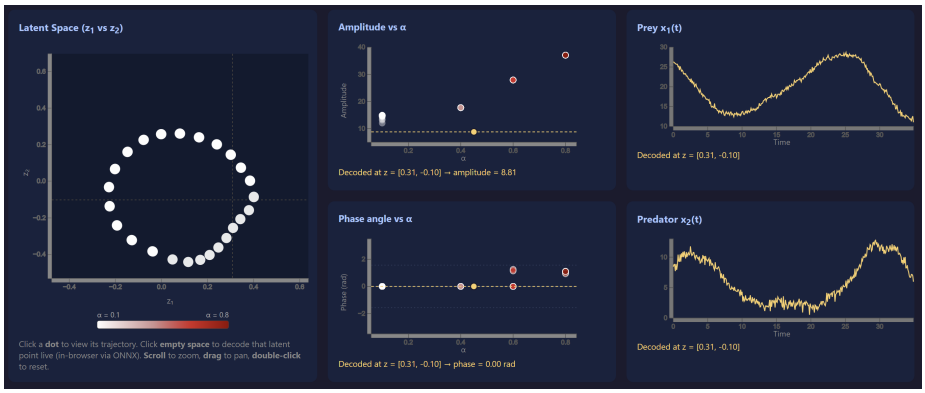

Figure 3: An interpolated point from the Lotka–Volterra predator–prey model α sweep

3.6 AI Use

We used AI to aid us in generating javascript code with d3, html code to visualize our generated datasets, and to seek ways to improve our VAE model architecture. To achieve this we created a schema from scratch, writing down with rigor how the data was sampled, how we expect it to be visualized, and what interactions we are interested in, this schema was updated in an iterative manner as to include new things added by the AI, and or any changes that were made to accomodate project needs. The model used throughout development was Claude's Opus 4.6 and Opus 4.7.

4 Dataset(s)

Our framework generates all data synthetically by numerically integrating the governing equations, providing complete control over ground-truth parameter labels and known bifurcation boundaries.

- **Lotka–Volterra predator–prey model**: governed by

$$\begin{aligned} \frac{dx_1}{dt} &= \alpha\, x_1 - \beta\, x_1 x_2, \\ \frac{dx_2}{dt} &= \delta\, x_1 x_2 - \gamma\, x_2, \end{aligned} \tag{3}$$

  we train **three** model variants, each holding three parameters fixed and varying one: the prey growth rate α, the predation coupling rate β, and

the predator death rate γ. These produce qualitatively distinct latent geometries despite sharing the same ODE structure, and serve as a controlled comparison of how different parameters shape the learned manifold.

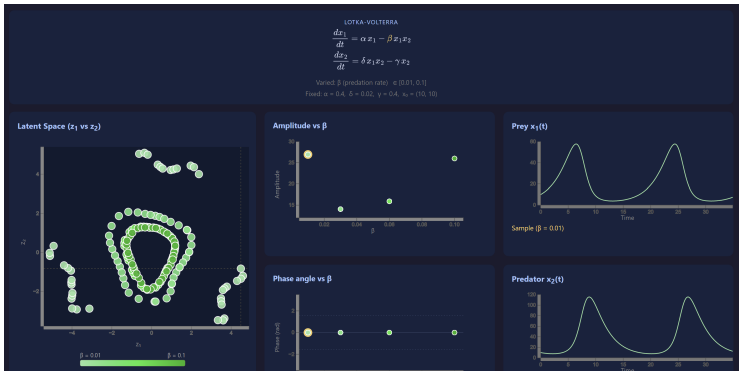

Figure 4: Lotka–Volterra predator–prey model - β sweep visualized

- **FitzHugh–Nagumo model**: a reduced model of neuronal excitability governed by

$$\begin{aligned} \frac{dv}{dt} &= v - \frac{v^3}{3} - w + I, \\ \frac{dw}{dt} &= \tau\,(v + a - b\,w). \end{aligned} \tag{4}$$

Two variants are trained: one varying the recovery bias a, which crosses a Hopf bifurcation and transitions the system between oscillatory and quiescent regimes; and one varying the injected current I within the purely oscillatory regime, encoding spiking frequency and amplitude changes.

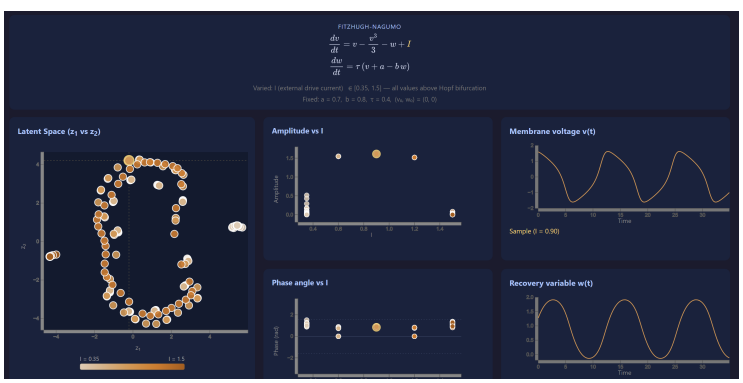

Figure 5: FitzHugh-Nagumo model - I sweep visualized

- **Duffing oscillator**: governed by

$$\frac{d^2x}{dt^2} + \delta\frac{dx}{dt} + \alpha x + \beta x^3 = \gamma\cos(\omega t), \tag{5}$$

we train **five** variants varying δ, α, β, γ, and ω, capturing changes in damping, linear stiffness, nonlinear stiffness, forcing strength, and phase respectively.

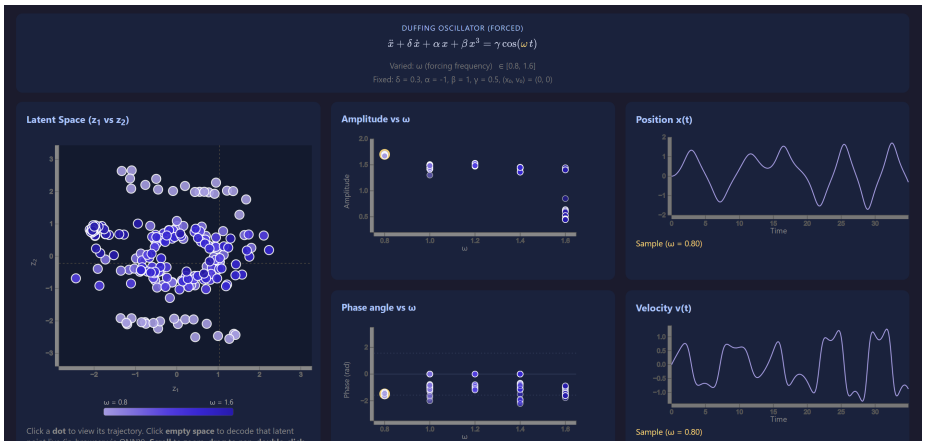

Figure 6: Duffing oscillator - ω sweep visualized

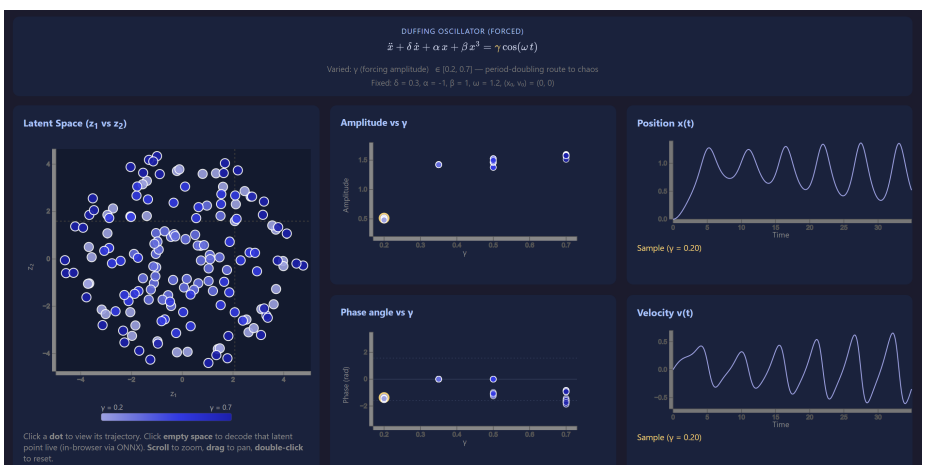

Figure 7: Duffing oscillator - γ sweep visualized

- **Van der Pol oscillator**: governed by

$$\frac{d^2x}{dt^2} - \mu(1 - x^2)\frac{dx}{dt} + \omega_0^2 x = \gamma \cos(\omega_f t), \tag{6}$$

we train **four** variants varying μ, ω_0, γ, and ω_f, encoding nonlinear damping strength, natural frequency, forcing amplitude, and forcing frequency.

Figure 8: Van Der Pol Oscillator- ω sweep visualized

- **Lorenz system**: governed by

$$\begin{aligned} \frac{dx}{dt} &= \sigma(y - x), \\ \frac{dy}{dt} &= x(\rho - z) - y, \\ \frac{dz}{dt} &= xy - \beta z, \end{aligned} \tag{7}$$

we train **three** variants varying σ, ρ, and β, capturing convection strength, chaotic regime transitions, and vertical dissipation.

An interesting note here is that the visualization is very misleading as **chaotic** ODE systems are very sensitive to starting input values so this system was visualized just to experiment and would not be included in the final version of the tool.

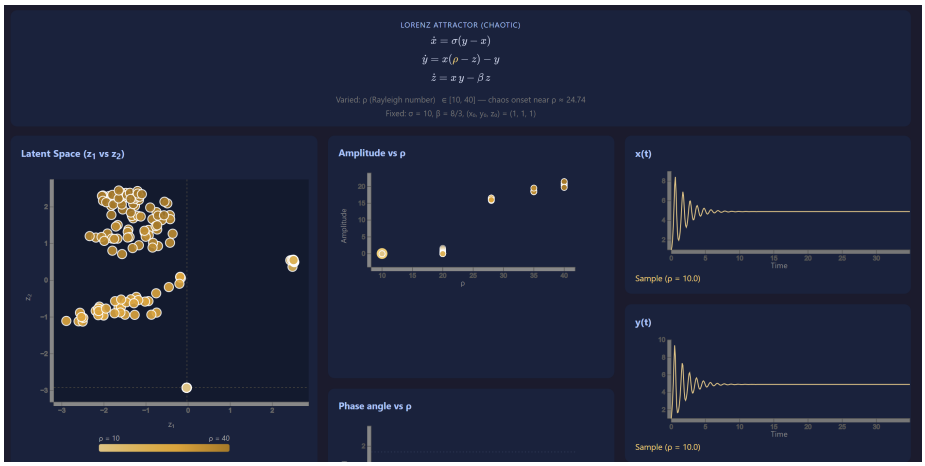

Figure 9: Lorenz Attractor - ρ sweep visualized

- **Brusselator model**: governed by

$$\begin{aligned} \frac{dx}{dt} &= A - (B+1)x + x^2 y, \\ \frac{dy}{dt} &= Bx - x^2 y, \end{aligned} \tag{8}$$

  we train **two** variants varying A and B, controlling inflow rate and autocatalytic feedback strength.

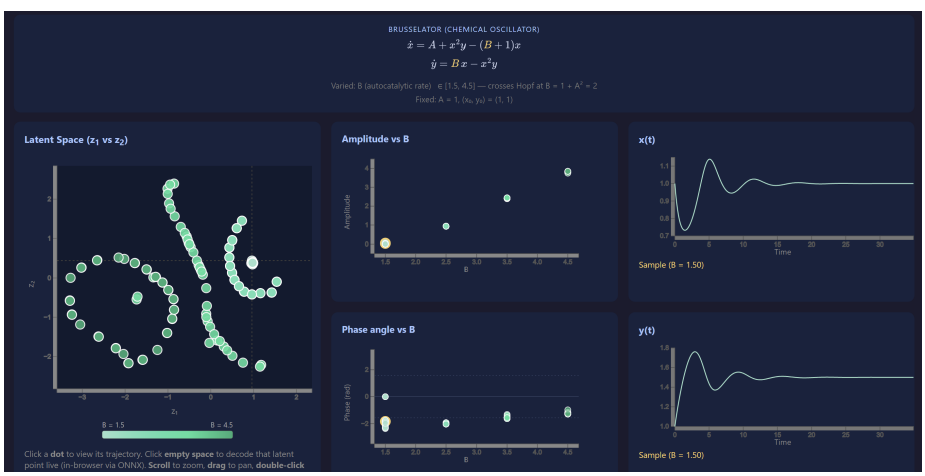

Figure 10: Brusselator Model - B sweep visualized

- **Selkov model**: governed by

$$\frac{dx}{dt} = -x + Ay + x^2 y,$$
$$\frac{dy}{dt} = B - Ay - x^2 y, \tag{9}$$

we train **two** variants varying A and B, controlling activation and inhibition balance in glycolytic oscillations.

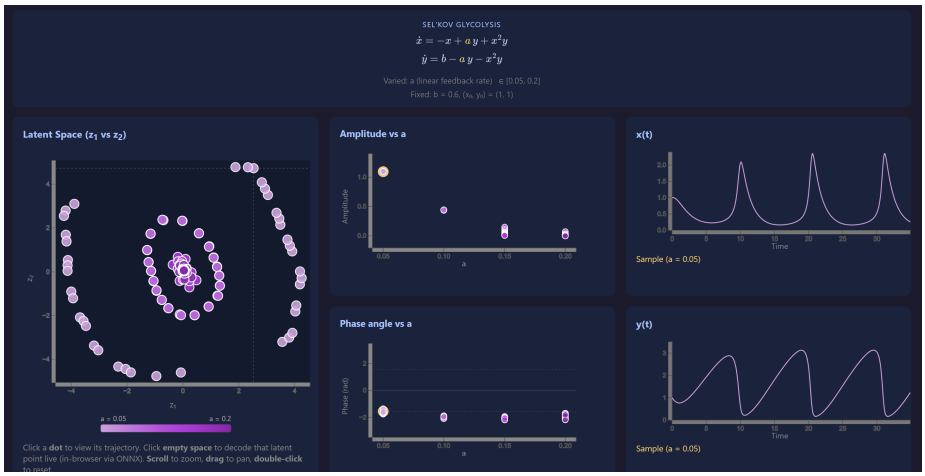

Figure 11: Selkov Model - A sweep visualized

Each model is trained on 800 windowed samples drawn from 20 parameter values, with trajectories integrated over $t \in [0, 100]$ using an adaptive Runge–Kutta 4/5 solver. All data generation, training, and export code is released alongside the tool here.

5 Evaluation Methods & Results

5.1 Results

We were able to train the VAE to produce a latent space in which trajectories with similar qualitative behavior (e.g., periodic, chaotic) form coherent clusters, and in which transitions between regimes appear as smooth gradients or sharp boundaries that correspond to known bifurcations. The interactive interface was able to help users to (1) identify qualitative behavioral regimes without manual trajectory inspection, (2) locate parameter-space boundaries associated with bifurcations, and (3) generate and inspect novel trajectories by navigating the latent space, thereby accelerating hypothesis formation about parameter–dynamics relationships.

5.2 Evaluation Methods

We evaluated the framework along three complementary axes.

Quantitative Evaluation.

- Training progress is monitored via the ELBO (Evidence Lower Bound), decomposed into its reconstruction term (MSE) and KL divergence, tracked separately on an 80/20 train/validation split across all 500 epochs.

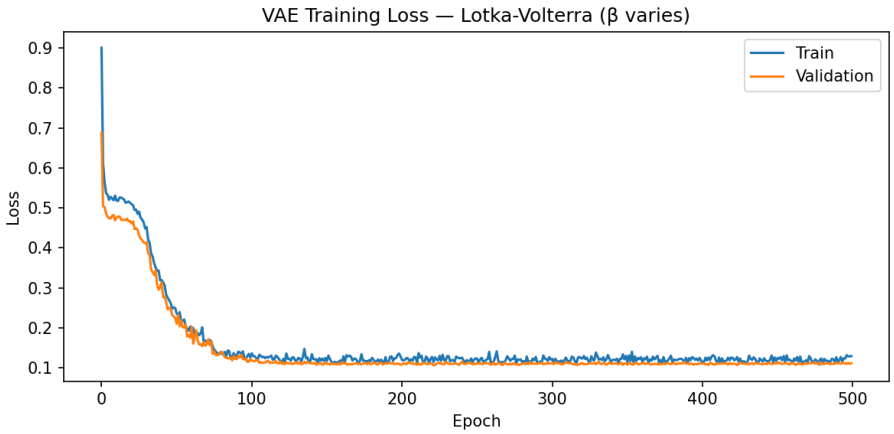

Figure 12: VAE Training Loss of the Lotka-Volterra Model as β Varies

- To assess whether the latent space organizes itself in a physically meaningful way, we compute the Pearson correlation between each latent dimension z_i and the known ground-truth parameter labels, identifying which axis carries the parameter signal and providing a scalar measure of disentanglement per model.

- Reconstruction fidelity on held-out (steps in the uniform training window that were skipped) and unseen (parameter values randomly placed in between pairs of training values) windows is reported as mean squared error in the normalized feature space.

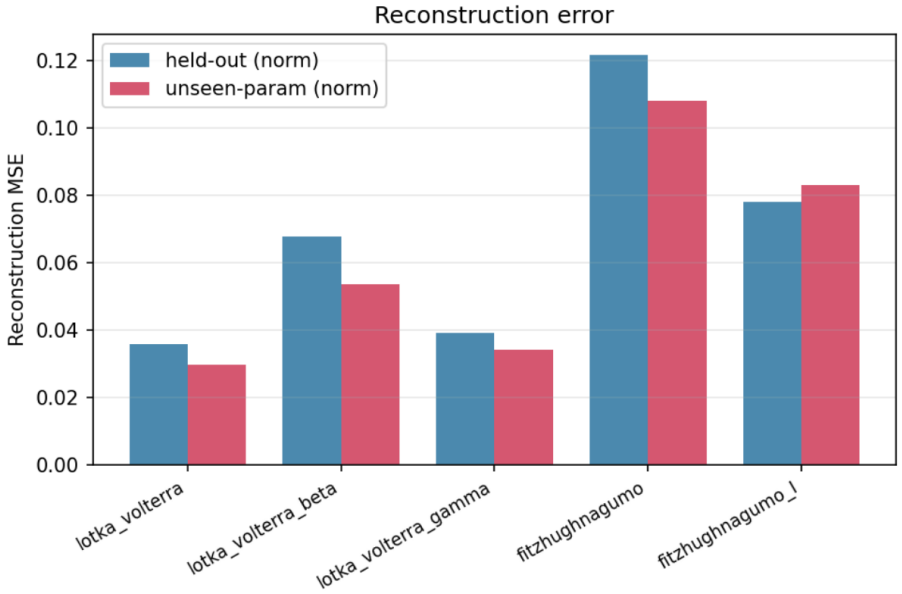

Figure 13: Relatively Low MSE (and similar levels across held-out and unseen values) indicate that our VAE is interpolating well (within our input domain)

Case Studies. We conduct two detailed case studies — one on the Lotka-Volterra model system and one on the FitzHugh-Nagumo model — demonstrating end-to-end workflows in which a user explores the latent space, identifies clusters, examines decoded trajectories, and relates findings to known dynamics from the literature on these ODEs.

For the Lotka-Volterra system 14, we observe in the latent space the formation of circular patterns that characterize the structure of trajectories across a representative domain of the predator-prey model. These structures reflect the inherently periodic nature of the system, where predator and prey populations evolve in cyclic dynamics. In particular, trajectories with similar oscillation amplitudes and phase relationships are embedded near one another, forming coherent rings in latent space.

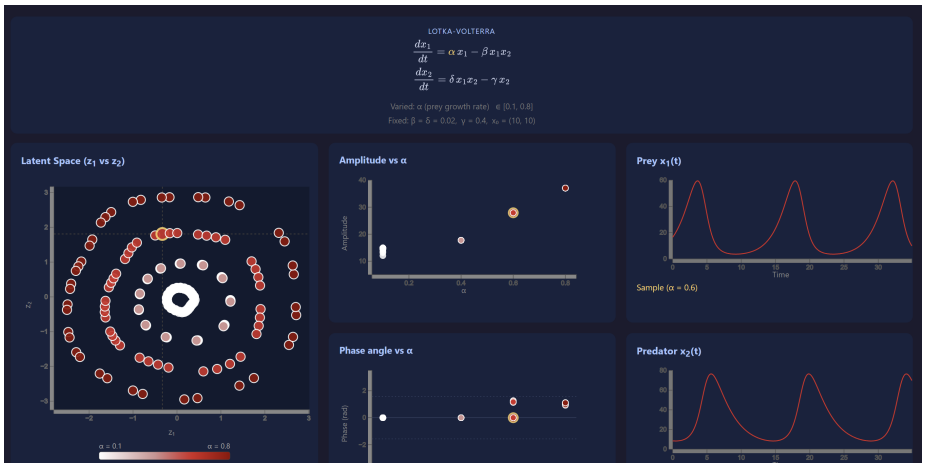

Figure 14: The circular patterns found in our visualization that we expected to find based on how this specific ODE works.

For the FitzHugh-Nagumo model, the latent space exhibits clear separation between excitable and oscillatory regimes. Regions corresponding to stable fixed-point behavior (2 external clusters) are distinct from those showing sustained oscillations (internal ellipse shape), and transitions between these regimes align with known bifurcation behavior in the literature. Decoded trajectories from intermediate latent regions show mixed or transitional dynamics, indicating that the model captures smooth interpolations between dynamical regimes.

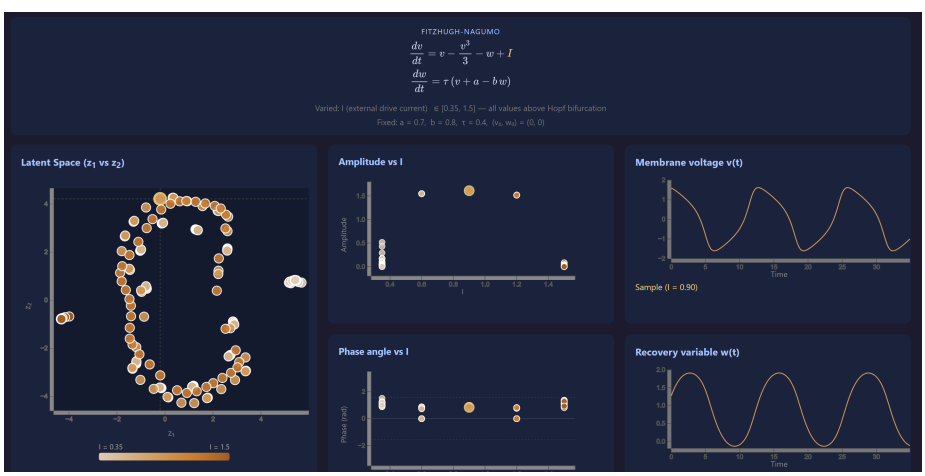

Figure 15: The 2 external clusters can be seen on the left and right of the central ellipse shape.

Overall, these case studies suggest that the learned latent space not only organizes trajectories in a visually interpretable manner, but also aligns with known qualitative properties of nonlinear dynamical systems.

Qualitative User Feedback. We reached out to several students with a mathematical background and received feedback from two of them. The feedback was collected after they used the system and compared what they saw to other ODE visualizations. Essentially, they performed a comparative analysis since they did not have in-depth knowledge of the ODE systems we had visualized.

- One user stated that our tool was more complete in its visualization since "It looks like it gets the system's dynamics throughout the whole domain and range worth looking at." Both users also thought that the visualizations were interesting to look at—"definitely interesting for students who see it for the first time"—but suggested that we could include some form of animation, similar to other ODE visualizations they had seen.

- The users were also unclear on what it meant for the visualization to exist in a two-dimensional latent space or what their coordinates meant when the underlying parameters of the ODE were higher-dimensional, especially since some other visualizations they compared against were displayed in three dimensions.

- Both users liked the simple interface, which they found easy to understand and use. One user questioned the validity of the extrapolation of data points outside our pre-calculated domain performed by the VAE. Another user stated that they "liked that the variable I am sweeping is highlighted," identifying this as a positive aspect of our visualization design.

- We also received valuable feedback from **Aiden Linder** at the end of our final presentation. He suggested that, for latent-space views with clear patterns, coordinates outside the pre-calculated domain should be made unclickable by the user, since the neural network's extrapolations become more unreliable the further you move outside the range of training data.

6 Limitations and Lessons Learned

We have some limitations:

- For a domain-expert to see whether their data follows the patterns represented by the visualizations shown, they still need to graph their own data and (possibly) reduce dimensions the same way we did.

- As of now, the extrapolated data points that are far outside the domain of our pre-trained set may not provide the most reliable information regarding amplitude, phase, or trajectories.

We also learned some valuable lessons, the most important being the value of user feedback and the **value of having many iterations of development and feedback**. More specifically:

- We made specific improvements to our visualization after we changed our random distribution of input data to a uniform distribution to make patterns in the latent space more visible. This improvement was based on feedback received from Taoseef Aziz after our Interim Presentation.
- It was interesting to note that while charting various ODEs, the chaotic Lorenz system looks like it is being visualized with no problem. However, thanks to our specific knowledge of the ODEs being visualized, we knew that it was misleading to a prospective user that was not aware of how chaotic ODEs work. Here, we learned the value of having in-depth knowledge of the data that is being visualized.
- We saw the value of user feedback in our work after specific suggestions like the uniform window suggestion from Taoseef and the limitation of the clickable Latent Space view suggestion by Aiden.

7 Future Work

We have several improvements that we believe could greatly improve the experience of using our tool, listed below.

- To help domain experts determine whether their data follows patterns represented by the visualizations shown, we propose adding a data-input section that allows users to upload their own data directly into the tool. This would enable the system to automatically detect similarities between the uploaded data and the learned latent-space structures, while also visualizing the user's data within the existing framework.
- Incorporating the suggestion from **Aiden Linder** would improve the usability of our system, although each ODE may require a custom unclickable region to be defined. Depending on the system, these regions may or may not be easy to compute within a generalized architecture. As a result, this approach could reduce the extensibility of our system and require additional custom work for each newly introduced ODE.
- Mixed parameter-sweep visualizations. These were already attempted, but did not provide any value in the 2-D visualization. We did think of potentially extending into 3-D representations, which could provide more expressive views of interactions between multiple varying parameters.
- One simple improvement that is common in other ODE visualizations is the animations of the trajectories seen on the right of the visualization.

- We also propose exploring adaptive windowing techniques for the different frequency ODE systems, allowing the model to better capture dynamics that evolve over longer/shorter time scales.

References

[1] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *Proceedings of the 2nd International Conference on Learning Representations (ICLR)*, 2014. Available: `https://arxiv.org/abs/1312.6114`

[2] L. van der Maaten and G. Hinton, "Visualizing Data using t-SNE," *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008. Available: `https://jmlr.org/papers/v9/vandermaaten08a.html`

[3] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, "Neural Ordinary Differential Equations," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018. Available: `https://arxiv.org/abs/1806.07366`

[4] Y. Rubanova, R. T. Q. Chen, and D. Duvenaud, "Latent Ordinary Differential Equations for Irregularly-Sampled Time Series," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019. Available: `https://arxiv.org/abs/1907.03907`

[5] M. Sedlmair, C. Heinzl, S. Bruckner, H. Piringer, and T. Möller, "Visual Parameter Space Analysis: A Conceptual Framework," *IEEE Transactions on Visualization and Computer Graphics*, vol. 20, no. 12, pp. 2161–2170, 2014. Available: `https://doi.org/10.1109/TVCG.2014.2346321`

[6] S. Liu, P.-T. Bremer, J. J. Thiagarajan, V. Srikumar, B. Wang, Y. Livnat, and V. Pascucci, "Visual Exploration of Semantic Relationships in Neural Word Embeddings," *IEEE Transactions on Visualization and Computer Graphics*, vol. 24, no. 1, pp. 553–562, 2018. Available: `https://doi.org/10.1109/TVCG.2017.2745141`

[7] L. McInnes, J. Healy, and J. Melville, "UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction," *arXiv preprint arXiv:1802.03426*, 2018. Available: `https://arxiv.org/abs/1802.03426`

[8] I. Higgins, L. Matthey, A. Pal, C. Burgess, X. Glorot, M. Botvinick, S. Mohamed, and A. Lerchner, "β-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework," in *Proceedings of the 5th International Conference on Learning Representations (ICLR)*, 2017. Available: `https://openreview.net/forum?id=Sy2fzU9gl`

[9] T. Denouden, "VAE Latent Space Explorer," 2017. [Online]. Available: `https://tayden.github.io/VAE-Latent-Space-Explorer/`